

Major Project Report on

Reinforcement Learning Powered Generative Models for User-Centric Recommendations

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

by

Ayush Deshmukh(211IT014)

Siddharth Kelkar(211IT067)

under the guidance of

Dr. Anand Kumar M.



DEPARTMENT OF INFORMATION TECHNOLOGY
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA
SURATHKAL, MANGALORE - 575025

April, 2025

DECLARATION

by the B.Tech. Student

I/We hereby *declare* that the Project Work Report entitled

.....

.....

which is being submitted to the **National Institute of Technology**

Karnataka, Surathkal for the award of the Degree of Bachelor of

Technology in

.....

is a *bonafide report of the work carried out by me/us*. The material contained in this Project Work Report has not been submitted to any University or Institution for the award of any degree.

Register Number, Name & Signature of the Student(s):

(1)

(2)

(3)

(4)

Department of

Place: NITK, SURATHKAL

Date:

C E R T I F I C A T E

This is to *certify* that the B.Tech. Project Work Report entitled

.....
..... submitted by :

Sl.No. Register Number & Name of Student(s)

(1)

(2)

(3)

(4)

as the record of the work carried out by him/her/them, is *accepted*

as the B.Tech. Project Work Report submission in partial fulfillment of

the requirements for the award of degree of **Bachelor of Technology**

in

Guide(s)

(Name and

Signature with Date)

Chairman - DUGC

(Signature with Date and Seal)

ABSTRACT

This research outlines a foundational approach for enhancing game sequence recommendations through advanced reinforcement learning and generative modeling techniques. The study utilizes a dataset from the Steam video game distribution network, which includes detailed user gaming histories and extensive game metadata. To enrich the feature set and improve recommendation accuracy, the approach incorporates custom-generated images derived from textual data such as reviews and genres.

The proposed framework consists of two primary components: the Recommendation Generator, which produces personalized game sequences; the User Preference Evaluator, which estimates the likelihood of user engagement. The model is pre-trained after which, Reinforcement Learning phase follows where the generator is optimized based on feedback from evaluator.

This work serves as a preliminary step, laying the groundwork for further development and refinement of a sophisticated game recommendation system. The goal is to deliver highly personalized gaming experiences by leveraging the synergy of Reinforcement Learning and Generative Adversarial Networks.

Keywords— Adaptive Recommendations, Context-Aware Systems, Generative Models, Multimodal, Reinforcement Learning, Text-to-Image Models

CONTENTS

LIST OF FIGURES	iv
LIST OF TABLES	iv
1 INTRODUCTION	1
1.1 Overview	1
1.2 Motivation	2
2 LITERATURE REVIEW	4
2.1 Background and Related Works	4
2.1.1 Evolution of Recommendation Systems	4
2.1.2 Reinforcement Learning Algorithms in Recommendation Systems	7
2.1.3 Frameworks and Models in Modern Recommendation Systems	7
2.2 Outcome of Literature Review	9
2.3 Problem Statement	9
2.4 Objectives of the Project	10
3 PROPOSED METHODOLOGY	11
3.1 Dataset Description	11
3.2 Custom Image Features	12
3.3 Architectural Overview	12
3.3.1 Rationality Discriminator and User Preference Evaluator . . .	14
3.3.2 Recommendation Generator	14
3.3.3 Modules of Generator and Discriminator	14
3.4 Training	17
3.4.1 Dual Discriminator Approach	17
3.4.2 Adapted Proximal Policy Optimization (PPO) Framework for Sequential Game Recommendations	18
4 Results and Discussions	24
4.1 Exploratory Data Analysis	24
4.2 Embedding	27
4.2.1 Game Embeddings-Method 1	27

4.2.2	Embedding Methodology-Method 1	28
4.2.3	User Embeddings	31
4.2.4	Remote Computing	33
4.3	Implementation of Generator and Discriminator	35
4.4	Implementation of Fixed Discriminator	36
5	CONCLUSIONS AND FUTURE WORK	46
	REFERENCES	48

LIST OF FIGURES

3.3.1 Proposed User Preference Evaluator/Rationality Discriminator	13
3.3.2 Proposed Recommendation Generator	23
4.1.1 Histogram Representing distribution of games	24
4.1.2 Top 20 most bought games	25
4.1.3 Top 20 most played games	26
4.1.4 CS:GO header	27
4.1.5 CS:GO in game images	27
4.2.1 Shell script used for generating embeddings	34
4.2.2 Output after submitting job to compute node	34
4.4.1 Critic Loss over Epochs	38
4.4.2 Actor Loss over Epochs	39
4.4.3 Average Reward over Epochs	40
4.4.4 Title Only vs Full Information	43
4.4.5 Dual vs Single Discriminator	44
4.4.6 Genre Only vs Full Metadata	45

LIST OF TABLES

4.2.1 Game Genres	32
4.2.2 Game Categories	33
4.2.3 Game sample example	33
4.3.1 User Feature Vectors Across 4 Gaming Categories	36
4.3.2 Sample Game Recommendations and Discriminator Confidence Scores	36

CHAPTER 1

INTRODUCTION

1.1 Overview

The systems that recommend have become an essential component of many digital platforms, significantly influencing how users discover content, products, and services. Recommendation systems are widely used to filter vast amounts of information and provide personalized suggestions that match individual user preferences. These systems enhance user experience by reducing information overload, improving customer satisfaction, and increasing engagement through relevant and timely recommendations. By analyzing historical data such as user behavior, preferences, and interactions, recommendation systems can predict what users are likely to be interested in, leading to higher conversion rates and better user retention. As a result, they play a critical role in driving business growth, improving user satisfaction, and fostering long-term customer relationships.

Traditional methods often rely on static user profiles or predefined rules, which do not adapt well to changing user preferences and are limited to optimizing short-term goals like click-through rates or immediate purchases. In contrast, RL models consider the recommendation process as a sequential decision-making problem, where each recommendation affects future user behavior.

This report investigates the integration of advanced Reinforcement Learning (RL) and generative models to enhance game recommendation systems. It focuses on how RL algorithms such as REINFORCE[1], Actor-Critic Methods[2], and Self-Critical Sequence Training[3] improve the stability and efficiency of recommendation processes, allowing systems to better balance immediate user engagement with long-term satisfaction.

The dataset for this research includes detailed user interaction records and game metadata from the Steam video game distribution network. To further enhance the recommendation process, custom-generated images based on textual and metadata attributes are utilized. The proposed model framework features a recommendation generator, a user preference evaluator, and a rationality discriminator, all driven

by RL to optimize recommendations. This work aims to be the stepping stone for developing a game recommendation system that surpasses current limitations and improves adaptability in personalized gaming experiences.

1.2 Motivation

The motivation behind this study stems from the pressing need to address the growing complexity of user preferences and the dynamic nature of game interactions in today’s rapidly evolving gaming landscape. As games become increasingly sophisticated and diverse, understanding user preferences has become a multifaceted challenge. Traditional recommendation systems often face challenges in balancing short-term user engagement with the use of advanced RL algorithms and generative models to maintain long-term user satisfaction. This struggle is particularly pronounced in the gaming context, where user preferences and behaviors can shift dramatically based on various factors, including trends, personal experiences, and social interactions.

In this intricate environment, there is a significant opportunity to enhance the relevance and effectiveness of game recommendations. By incorporating detailed game metadata—such as genres, platform compatibility, game duration, multiplayer vs. single-player modes, and unique gameplay mechanics—we can capture the essential attributes of games that resonate with different user segments. For example, a user who enjoys action-packed multiplayer games may have different preferences compared to someone who favors narrative-driven single-player experiences. Understanding these distinctions allows us to create more nuanced recommendations that are tailored to specific user profiles, thus improving the likelihood of user engagement and satisfaction.

Additionally, leveraging user historical data, such as previous interactions, ratings, and preferences, can enrich the recommendation process. By analyzing this data, we can identify patterns and trends in user behavior that can inform future recommendations. For instance, if a user frequently plays cooperative games with friends, the system can prioritize similar multiplayer titles or suggest games that enhance social interaction. This personalized approach has the potential to significantly improve overall gaming experiences by providing users with suggestions that resonate more

closely with their interests and preferences.

This research aims to explore how these innovative technologies can be integrated into a robust and adaptive recommendation system that evolves alongside user preferences. Such a system would not only meet current user needs but also anticipate future preferences based on emerging trends and gameplay patterns, thereby enhancing the overall relevance of the recommendations. By employing advanced techniques in machine learning and data analysis, the system could continuously learn from user interactions, adapting its suggestions in real-time to align with changing interests.

Ultimately, the goal is to create a more engaging, satisfying, and holistic gaming environment that enhances user retention and enjoyment. By setting a new standard in personalized gaming experiences, this research has the potential to transform how users interact with games, fostering a deeper connection between players and the gaming content they consume. This transformative approach not only prioritizes immediate satisfaction but also nurtures long-term engagement, ultimately contributing to a more vibrant and sustainable gaming community.

The rest of the report is organized as follows. Chapter 2 reviews relevant literature on recommendation systems, their evolution, RL algorithms, and modern frameworks, concluding with our problem statement and project objectives. Chapter 3 details our proposed methodology, including dataset description, custom image features, and architectural components of our recommendation system. Chapter 4 presents results and discussions, covering data analysis, embedding methods, and implementation details of our generator-discriminator model. Finally, Chapter 5 concludes the work and suggests directions for future research.

CHAPTER 2

LITERATURE REVIEW

2.1 Background and Related Works

Recommendation systems are intelligent algorithms designed to predict user preferences and suggest relevant items or content. These systems have become integral in today’s digital landscape, where users face an overwhelming amount of choices across various platforms. From e-commerce sites recommending products to streaming services suggesting movies, recommendation systems help filter through vast amounts of information to present users with personalized options. Improving the user experience is the main objective, by reducing search time, increasing engagement, and improving satisfaction. For businesses, effective recommendation systems can lead to increased sales, user retention, and a competitive edge in the market.

The development of recommendation systems will be examined in the parts that follow, beginning with early approaches and moving on to more sophisticated strategies. This review begins by examining traditional approaches such as collaborative filtering and content-based methods, which laid the foundation for modern recommendation systems. The discussion then delves into the impact of deep learning on the field, showcasing how neural networks have revolutionized recommendation algorithms. Subsequently, attention-based approaches are discussed, which have significantly improved the ability to capture complex user-item interactions. Finally, the review explores the emerging trend of Reinforcement Learning (RL) in recommendation systems, highlighting how these methods can adapt and optimize recommendations in real-time based on user feedback and behavior.

2.1.1 Evolution of Recommendation Systems

A key strategy in recommendation systems is Collaborative Filtering (CF)[\[4\]](#), which examines user-item interactions to produce tailored recommendations. A key strategy in recommendation systems is Collaborative Filtering (CF)[\[4\]](#), which examines user-item interactions to produce tailored recommendations. This approach has shown

to provide better performance and quality than user-based algorithms, especially for large-scale problems. The item-to-item collaborative filtering algorithm employed by Amazon [5] is another noteworthy development in CF. It generates high-quality recommendations in real-time for large datasets and scales irrespective of the number of customers and products.

To further improve CF methods, researchers have explored various techniques. Rendle et al.[6] proposed an adaptive regularization method that automatically adjusts regularization parameters during model training, eliminating the need for time-consuming hyperparameter searches. Barkan and Koenigstein[7] introduced item2vec, a neural embedding algorithm inspired by word2vec from natural language processing, which learns item representations in a latent space without requiring user information. These advancements have significantly enhanced the effectiveness and efficiency of collaborative filtering techniques in recommendation systems.

Wang et al. more recently presented Neural Graph Collaborative Filtering (NGCF)[8], which overcomes a significant drawback in the current embedding-based CF techniques.. Traditional approaches typically derive user and item embeddings from pre-existing features, potentially overlooking the collaborative signal inherent in user-item interactions. NGCF explicitly encodes collaboration signals by incorporating the user-item interaction graph structure into the embedding procedure. More expressive user and item representations result from NGCF’s efficient modeling of high-order connectivity through the propagation of embeddings on the user-item graph. The significance of embedding propagation in capturing collaborative effects for recommendation tasks was highlighted by extensive trials on public benchmarks, which showed notable gains over state-of-the-art models.

Deep learning has revolutionized the field of recommendation systems by enabling more complex and expressive models. The ability of deep convolutional neural networks to recommend music was shown by Van den Oord et al.[9], who outperformed conventional methods in predicting latent features from audio inputs. This method is particularly useful for recommending new and unpopular music, addressing the new user problem common in filtering collaboratively.

The Collaborative Deep Learning (CDL) model was proposed by Wang et al.[10]. It combines collaborative filtering for the ratings matrix with deep representation

learning for content information. By utilizing supplementary information like item content, this method successfully solves the sparsity issue in traditional CF techniques. Similar to this, Elkahky et al.[11] presented a multi-view deep learning method for cross-domain user modeling that represents users using rich feature sets derived from their search queries and online browsing histories. This method significantly improves recommendation quality across different domains and scales well to large datasets.

Recurrent Neural Networks (RNNs) is also applied to recommendation tasks, particularly for session-based recommendations. Hidasi et al.[12] proposed an RNN-based approach that models the whole session to provide more accurate recommendations, outperforming traditional item-to-item recommendation methods.

Attention mechanisms have gained significant traction in recommendation systems due to their ability to capture fine-grained user-item interactions and context-aware information. The Social Attentional Memory Network (SAMN), first presented by Chen et al.[13], tackles the different friend-level impacts and aspect-level preferences in socially aware recommendations. This model adaptively chooses informative friends for user modeling by using a friend-level attention component and an attention-based memory module to learn user-friend relation vectors.

In order to properly capture item-item correlations from user interaction trajectories, Zhang et al.[14] presented a self-attention based model for next-item recommendation. In order to learn more accurate representations of users' fleeting interests, this method shows how self-attention can be used to estimate the relative value of each item in user histories.

In the context of heterogeneous information networks (HIN), Hu et al.[15] developed a neural co-attention model that explicitly incorporates meta-path based context for top-N recommendations. With better performance and interpretability, this model develops useful representations for users, items, and context based on meta-paths.

Dual graph attention networks were introduced by Wu et al.[16] to describe complex social effects in recommender systems. Using a policy-based fusion mechanism to weigh interactions of different social effects, this method jointly learns representations for two-fold social effects in both the user and item domains.

2.1.2 Reinforcement Learning Algorithms in Recommendation Systems

RL has emerged as a powerful paradigm in recommendation systems, enabling more adaptive and dynamic recommendations. Several key algorithms and approaches have been developed to leverage RL in this domain:

REINFORCE[1] and Actor-Critic Methods[2]: These algorithms have shown particular efficacy in optimizing the trade-off between immediate user engagement and long-term satisfaction metrics in recommendation systems. The REINFORCE algorithm uses direct policy gradient optimization, while Actor-Critic architectures combine value estimation with policy optimization for more stable and efficient learning processes. These methods have proven highly effective in recommendation systems, synergistically combining value estimation with policy optimization to yield more stable and efficient learning processes.

Self-Critical Sequence Training (SCST): This approach, developed by Rennie et al. [3], has demonstrated considerable efficacy in optimizing entire recommendation sequences. By improving the overall alignment with user interests and preferences, SCST enhances the quality of sequential recommendations. It has shown particular effectiveness in the domain of sequence generation, optimizing outputs relative to a learned baseline.

2.1.3 Frameworks and Models in Modern Recommendation Systems

Deep Q-Networks for Ad Placement: Xiangyu et al. [17] applied Deep Q-Networks to optimize ad placement in recommendation systems. Their approach estimates Q-values to effectively balance ad revenue and user experience, demonstrating the potential of RL in multi-objective optimization scenarios. This method has shown significant promise in balancing the competing objectives of maximizing ad revenue while maintaining a positive user experience.

Wide&Deep and DeepFM Models: While not strictly RL methods, these models by Cheng et al. [18] and Guo et al. [19] respectively, combine deep learning with factorization machines to improve recommendation accuracy. They represent important

foundational techniques that complement RL approaches in recommendation systems. These models have successfully combined deep learning with factorization machines to significantly improve recommendation accuracy, providing a solid foundation for more advanced RL-based approaches.

Behavior Pattern Mining-based Multi-behavior Recommendation (BPMR): Introduced by Li et al. [20], it utilizes a Bayesian approach to mitigate challenges such as over-smoothing in graph neural networks, significantly improving recommendation accuracy. BPMR overcomes issues in traditional methods by deeply exploring diverse user-item interaction patterns and leveraging behavior patterns as features. It employs a Bayesian approach to mitigate challenges such as over-smoothing in graph neural networks. By analyzing behavior patterns based on their length and combinations, and using paths as features for more targeted recommendations, BPMR has demonstrated substantial improvements in recommendation accuracy. Reported results show average enhancements of 268.29

ROLeR: Zhang et al. [21] proposed this model-based approach to refine reward and uncertainty estimation in offline RL for recommender systems. ROLeR innovates through Reward Shaping and Uncertainty Penalty techniques, providing a more robust alternative to traditional ensemble approaches for uncertainty estimation in offline RL scenarios. This method addresses the prevalent challenge of discrepancies between offline data and real-world interactions, which often leads to inaccurate reward estimates and uncertain predictions. ROLeR introduces two main strategies: Reward Shaping, a non-parametric technique that enhances reward estimation by clustering users with similar preferences and inferring rewards from nearest neighbors, and Uncertainty Penalty, a versatile penalty mechanism based on the quality of clustering. Demonstrated on four benchmark datasets, ROLeR shows substantial improvements in recommendation performance, marking a notable advancement in the field of offline RL for recommender systems.

In conclusion, recent advancements in RL and behavior pattern mining have significantly improved recommendation systems. New approaches have enhanced how these systems adapt to user preferences and make better recommendations. Innovations like ROLeR have also refined how these systems estimate rewards and handle uncertainties, leading to more accurate and effective recommendations. Overall, these

developments represent a major step forward in creating smarter and more responsive recommendation systems.

2.2 Outcome of Literature Review

1. Most existing recommendation systems focus on optimizing for short-term user engagement or immediate metrics, without considering long-term user satisfaction and loyalty.
2. Existing models do not adequately capture the evolving nature of user preferences and the dynamic context in which recommendations are made. Adaptability and personalization remain key challenges.
3. While attention-based methods have shown promise, there is room for further advancements in capturing nuanced, context-aware relationships between users and items.

2.3 Problem Statement

The project aims to create a Generative Game Recommendation System that suggests personalized game sequences based on a user’s history and preferences. Using RL, the system balances immediate satisfaction with long-term engagement, ensuring game recommendations are both relevant and logically sequenced to enhance the user’s gaming experience.

For each user u , their historical record can be formulated as $U = \{G, f\}$, where:

- G is the game interaction record,
- f is the user profile feature vector, and

The game metadata(G_m) is used to:

- Improve the relevance of the recommended game sequences by considering attributes such as genre, price, and other metadata, ensuring that the recommended games align with the user’s interests and preferences.

Based on the above, the problem of game sequence recommendation can be formulated as follows:

Game Sequence Recommendation. Given a user u with the corresponding historical record U , game metadata G_m , and a number k , the goal of game sequence recommendation is to recommend a set S consisting of k games arranged in a sequence for user u . This sequence should be optimized based on the user’s preferences, interactions, feedback, and game metadata to enhance their overall gaming experience.

2.4 Objectives of the Project

- Customise recommendations to individual user preferences and historical behaviors to provide personalized gaming experiences.
- Utilize contextually relevant features from game images to enhance the relevance of the recommendations.
- Incorporate detailed game metadata (e.g., genres, mechanics, play styles) to enhance the relevance of game recommendations.
- Apply Generative Adversarial Networks (GANs) in conjunction with RL techniques to create more robust and adaptive game recommendation systems.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 Dataset Description

We use data from the Steam video game[22][23][24] distribution network for training purposes. The dataset offers a detailed snapshot of a specific Steam user’s gaming library and activity. In total, this user dataset has a data of 88,310 gamers and 10,978 games they purchased. It includes the user’s unique Steam identifier and a direct URL to their Steam profile, facilitating easy access and verification of the user’s gaming information. The core of the dataset is a comprehensive list of games owned by the user, where each game entry is characterized by several key attributes:

- **item-id:** A unique numerical identifier for each game, which can be used to reference the game within Steam’s ecosystem or integrate with other databases and services
- **item-name:** The official title of the game, providing clear identification and aiding in categorization or further analysis.
- **playtime-forever:** This metric records the total amount of time, measured in minutes, that the user has spent playing the game since its purchase. It serves as an indicator of the user’s long-term engagement and interest in the game.

By aggregating these attributes, the dataset enables a multifaceted analysis of the user’s gaming habits. For instance, comparing playtime-forever with playtime-2weeks can reveal trends such as sustained interest, resurgence in gameplay, or declining engagement with specific titles. Additionally, the diversity and range of games listed can provide insights into the user’s genre preferences, gaming behavior, and potential areas for personalized recommendations

The dataset also contains game metadata information for 32135 games with features as follows:

- **title:** Indicates the name of the game.

- **id:** A unique identifier for the game on Steam
- **url:** The URL to the game’s store page on Steam, where users can view more information
- **release-date:** The official release date of the game
- **price:** Indicate the original price of the game
- **genres and tags:** These lists categorize the game into specific genres and tags, such as Action, Casual, Indie, Simulation, and Strategy, which help in describing the gameplay and content.
- **reviews-url:** A link to the game’s review section on Steam, where users can read and browse through the most recent reviews.
- **specs:** This field lists specific features of the game, such as being a ”Single-player” game.

3.2 Custom Image Features

Apart from the existing features provided in the dataset, we aim to enhance the feature set of the games by incorporating images generated from textual features such as reviews, genres, and other relevant metadata. By generating multiple context-specific images for a game, we can create a more diverse and comprehensive feature set. This approach not only captures the textual nuances in a visual format but also adds a new dimension to the data, making the model more robust. The inclusion of these custom-generated images can potentially improve the model’s ability to understand and predict game-related outcomes by leveraging the rich information embedded in both text and images.

3.3 Architectural Overview

We will provide a detailed introduction to our model’s framework in this part. Our system is primarily composed of two parts, the User Preference Evaluator and the Recommendation Generator, as seen in Figures 3.3.1 and 3.3.2. Given a specific user, the

recommendation seeks to provide a list of items, and the user performance evaluator seeks to determine the likelihood that the user will find the created recommendations appealing. Prior to beginning the training procedure, we pre-train the rationality discriminator and the preference evaluator. Following that, the RL technique uses the weighted sum of the outputs from the performance predictor and the rationality discriminator—two reward models—to train the generator. Details of the model are explained in the following subsections.

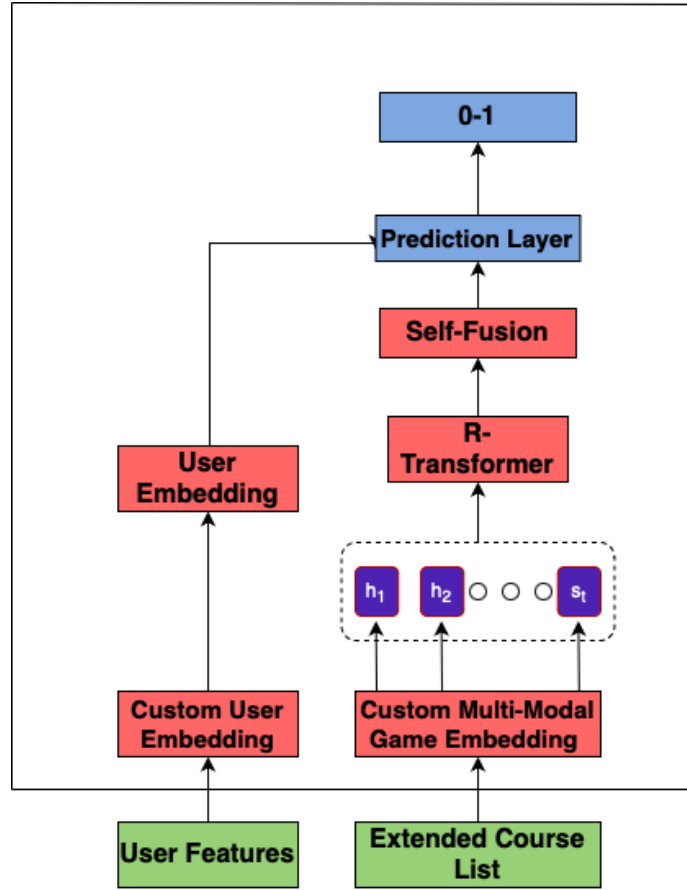


Figure 3.3.1: Proposed User Preference Evaluator/Rationality Discriminator

3.3.1 Rationality Discriminator and User Preference Evaluator

Our RL-based system consists of two reward models: the rationality discriminator and the User Preference Evaluator. These two models have exactly the same input and output formats. In particular, a real number between 0 and 1 is the output, and the input consists of the user’s profile information and gaming history. While the rationality discriminator confirms that the recommendations are reasonable—that is, that the created games are compatible with user preferences and not random—the user preference evaluator assigns a score to the generated suggestions based on the likelihood that the user will find them appealing.

3.3.2 Recommendation Generator

For the learning plan generator, the inputs are user features and game features. The generator needs to generate k games for user where k is the preset number. The generator is part of GAN paradigm with the rationality discriminator. In particular, we initially utilize the generator to obtain the extended record $L1$ given a user and his or her historical record L . And using the information provided by the employee, our rationality discriminator seeks to differentiate between $L1$ and L .

3.3.3 Modules of Generator and Discriminator

RG-Transformer

The RG-Transformer’s implementation of R-Attention (Relative Position-aware Attention) represents a crucial modification to standard self-attention mechanisms by introducing a relative positional bias term before the softmax operation. This enhancement allows the model to explicitly capture positional relationships between elements in a sequence, expressed mathematically as

$$\text{R-Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + B \right) V \quad (3.1)$$

where B represents the relative positional bias matrix that enables the model to maintain awareness of sequential dependencies and account for prerequisite relationships in the game structure.

Gated Cross Attention

The Gated Cross-Attention (GCA) layer serves as a crucial bridging mechanism that facilitates the interaction and information fusion between the historical sequence L (containing records $h_1, h_2, \dots$) and current generated sequence matrix S_t from the generator. At time step t , our generator produces a course sequence $s_t = \langle s_1, \dots, s_t \rangle$, which is converted into a title embedding matrix S_t and processed through RG-Transform as $S_t = \text{RG-Transform}(S_t)$. By implementing a structure similar to the RG-Transformer but with cross-attention characteristics, the GCA layer enables the model to effectively collate and synthesize information from both sources - the embedded historical sequences ($L = \text{RG-Transform}(\langle h_1, h_2, \dots \rangle)$) and the generator's transformed current sequence (S_t). This dual-source integration is achieved through a gated mechanism expressed as:

$$\text{GCA}(S_t, L) = S_t + \sigma(\text{MLP}(S_t)) \odot \text{LN}(\text{MLP}(S_t)) \quad (3.2)$$

$$S_t'' = S_t + \sigma(\text{MLP}(S_t)) \odot S_t' \quad (3.3)$$

$$S_t' = \text{MHA}(\text{LN}(S_t), \text{LN}(L), \text{LN}(L)) \quad (3.4)$$

where the Multi-Head Attention (MHA) component processes the normalized representations of both the historical sequence embedding (L) and generator's current time step sequences (S_t), while the MLP structure controls the flow of information, ensuring that relevant patterns from both sources are preserved and meaningfully combined in the final output.

Self-Fusion

The Self-Fusion mechanism implements a sophisticated approach to course embedding by leveraging a transformer-based architecture that combines self-attention and gating mechanisms. At its core, the process begins with layer normalization and multi-head

attention operations on the input tensor T , followed by MLP transformations:

$$T_i = \text{LN}(T' + \text{MLP}(T')) \quad (3.5)$$

$$T' = \text{LN}(T + \text{MHA}(T, T, T)) \quad (3.6)$$

The Self-Fusion mechanism applies an additional self-attention operation to refine and consolidate the combined representations from GCA:

$$s_t = \text{Self-Fusion}(\text{GCA}(S_t, L)) \quad (3.7)$$

This final transformation enables the model to perform thorough contextualization where each element in the sequence attends to all other elements, effectively capturing higher-order relationships and dependencies that emerge from the fusion of historical $(h_1, h_2, \dots)$ and generated patterns.

The course embedding t_i is refined through a Self-Fusion layer that incorporates a gating mechanism:

$$t_i = \text{Self-Fusion}(T_i) \quad (3.8)$$

$$= \text{softmax}(\tanh(T_i W_a + v_a) W_g)^\top T_i \quad (3.9)$$

where $W_a \in \mathbb{R}^{d \times d}$, $v_a \in \mathbb{R}^d$, and $W_g \in \mathbb{R}^{d \times 1}$ represent trainable parameters that control the attention and gating mechanisms.

Prediction Layer

The final layer of the architecture performs prediction by concatenating the Self-Fusion output with user profile embeddings to generate game recommendations. The concatenated representation is processed through a prediction layer that maps to the game embedding space. This can be formally expressed as:

$$c_t = [s_t \parallel u] \quad (3.10)$$

$$g_t = \text{MLP}(c_t) \quad (3.11)$$

$$\hat{y} = \text{argmax}_{g \in \mathcal{G}} \text{sim}(g_t, g) \quad (3.12)$$

where:

- s_t is the output from the Self-Fusion layer incorporating both S_t and L
- u represents the user profile embeddings
- c_t is the concatenated representation
- g_t is the predicted game embedding
- $\mathcal{G}$ is the set of all game embeddings
- $\text{sim}(\cdot, \cdot)$ is a similarity function (e.g., cosine similarity)
- $\hat{y}$ is the final predicted game

The similarity between the predicted embedding g_t and the actual game embeddings in the database is computed, and the game with the highest similarity score is selected as the final recommendation. This approach enables the model to map the user’s historical interactions and preferences to the most suitable game in the embedding space.

3.4 Training

3.4.1 Dual Discriminator Approach

The second approach introduces a more nuanced architecture employing two discriminators, each serving a distinct purpose in the recommendation process. This method begins by establishing a clear definition of user preferences based on gameplay patterns. We consider a game to be "liked" by a user if their playtime exceeds the average playtime threshold (θ), expressed as:

$$Like(game) = \begin{cases} 1 & \text{if } playtime > \theta \\ 0 & \text{otherwise} \end{cases} \quad (3.13)$$

The first discriminator (D_1) is pre-trained on this user preference data and, crucially, maintains fixed weights throughout the remainder of the training process.

Working in tandem with D_1 , the second discriminator (D_2) undergoes continuous training within the GAN paradigm, focusing specifically on evaluating whether users will enjoy the generated recommendations and if the generated list of games is in sync with the historical games played.

3.4.2 Adapted Proximal Policy Optimization (PPO) Framework for Sequential Game Recommendations

We present a novel adaptation of the PPO algorithm specifically designed for sequential game recommendation generation. Our framework introduces three major innovations that enhance the traditional PPO algorithm to better handle the unique challenges of game recommendation sequences.

Enhanced PPO Core Mechanism

The foundation of our approach builds upon the standard PPO algorithm, but with significant modifications for sequential recommendation generation. At its core, we utilize the clipped surrogate loss function:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (3.14)$$

This objective function operates through several carefully designed components:

The policy probability ratio $r_t(\theta)$ measures the relative change in action probabilities:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \quad (3.15)$$

We employ Generalized Advantage Estimation (GAE) to compute advantages:

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l} \quad (3.16)$$

The temporal difference error δ_t is calculated as:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (3.17)$$

Innovative Dual Discriminator Reward Architecture

A key innovation in our framework is the dual discriminator reward mechanism that combines quality assessment with adaptability. The reward at each timestep is computed as:

$$r_t = \underbrace{\beta D_{fix}(u, G_{1:t})}_{\text{Fixed quality assessment}} + \underbrace{(1 - \beta) D_{train}(u, G_{1:t})}_{\text{Adaptive realism score}} \quad (3.18)$$

This sophisticated reward structure serves multiple purposes:

- D_{fix} : A pretrained discriminator with frozen weights that ensures consistent recommendation quality standards
- D_{train} : A trainable discriminator that continuously adapts to evolving user preferences and gameplay patterns. This acts as the Value estimate from the critic netw
- $\beta = 0.7$: Carefully tuned balance factor between fixed and adaptive reward components

Sequential Generation Process

Our generation process follows a structured sequence of steps at each timestep t :

1. **State Composition:** The state s_t is constructed as a rich representation:

$$s_t = \{u, H_{1:400}, G_{1:t-1}\} \quad (3.19)$$

where u represents user features, $H_{1:400}$ captures historical gameplay, and $G_{1:t-1}$ contains previously generated recommendations.

2. Action Generation:

$$a_t = \text{Actor}(s_t) \quad (3.20)$$

The transformer network generates game recommendation g_t based on the current state.

3. Sequence Update:

$$G_{1:t} = [G_{1:t-1}, g_t] \quad (3.21)$$

The generated sequence is updated to include the new recommendation.

4. Reward Computation: The dual discriminator ensemble evaluates the updated sequence to provide the reward r_t .

$$\mathcal{L}_G = \mathcal{L}_{PPO} + \lambda_1 \mathcal{L}_{Ent} \quad (3.22)$$

Generator (Actor) Loss Architecture

Our generator loss function combines three carefully designed components:

1. PPO Clip Loss:

$$\mathcal{L}_{PPO} = -\frac{1}{K} \sum_{k=1}^K \left[\min \left(r_k \hat{A}_k, \text{clip}(r_k, 1 - \epsilon, 1 + \epsilon) \hat{A}_k \right) \right] \quad (3.23)$$

This component ensures stable policy updates while maintaining performance improvements.

Algorithm 1 PPO-GAN Sequential Training Protocol

Input: User dataset $\mathcal{D}$, pretrained discriminator D_{fix} , hyperparameters

Output: Trained generator G_θ , trained discriminator D_ϕ

Initialize G_θ, D_ϕ Initialize experience buffer $\mathcal{B}$ **for** $epoch = 1$ **to** N_{epochs} **do**

for $batch \mathcal{X}_b \in \mathcal{D}$ **do**

 // Phase 1: Data Preparation

$\mathcal{X}_b \leftarrow \text{DictStateDataset} \triangleright$ Contains:

 - Static features $\mathbf{u}_i \in \mathbb{R}^{449}$

 - Game history $\mathbf{H}_i \in \mathbb{R}^{400 \times 2304}$

 - Generated games $\mathbf{G}_i^{gen} \in \mathbb{R}^{10 \times 2304}$

 // Phase 2: Trajectory Collection

 Initialize $\mathbf{G}_i^{gen} \leftarrow$ zero padding **for** $step t = 1$ **to** 10 **do**

 // State Composition

$\mathbf{s}_t \leftarrow \text{Concat}[\mathbf{u}_i, \mathbf{H}_i, \mathbf{G}_{1:t-1}^{gen}]$

 // Generator Forward Pass

$g_t, V_\theta(\mathbf{s}_t) \leftarrow G_\theta(\mathbf{s}_t), D_\phi(\mathbf{s}_t)$ Sample action $g_t \sim \pi_\theta$

 Append game $\mathbf{g}_t$ to $\mathbf{G}_i^{gen}$

 // Reward Calculation

$r_t^{fix} \leftarrow D_{fix}(\mathbf{u}_i, \mathbf{G}_t^{gen})$

$r_t^{train} \leftarrow D_\phi(\mathbf{u}_i, \mathbf{G}_{1:t}^{gen})$

$r_t \leftarrow \beta r_t^{fix} + (1 - \beta) r_t^{train}$

 Store $(\mathbf{s}_t, g_t, r_t, \pi_\theta(g_t|\mathbf{s}_t))$ in $\mathcal{B}$

 // Phase 3: Advantage Calculation

 Compute returns $R_t = \sum_{k=t}^{10} \gamma^{k-t} r_k$ Compute advantages $\hat{A}_t = R_t - V_\theta(\mathbf{s}_t)$ using GAE

 // Phase 4: PPO Updates

for $ppo_step = 1$ **to** 5 **do**

 // Generator Update

$\mathcal{L}_G \leftarrow \frac{1}{K} \sum_{k=1}^K \left[\min(r_k \hat{A}_k, \text{clip}(r_k, 1 - \epsilon, 1 + \epsilon) \hat{A}_k) \right] + \lambda_1 H(\pi_\theta)$ Update G_θ

 using $\nabla \mathcal{L}_G$ with Adam ($\alpha = 0.0002$)

 // Discriminator Update

$\mathcal{L}_D \leftarrow -\text{SmoothL1}(R_t - V_\theta(\mathbf{s}_t))$ with Adam ($\alpha = 0.0002$)

2. Entropy Regularization:

$$\mathcal{L}_{Ent} = - \sum_{a \in \mathcal{A}} \pi(a|s) \log \pi(a|s) \quad (3.24)$$

Promotes exploration and prevents premature convergence to suboptimal policies.

Discriminator (Critic) Architecture

The value loss function in our PPO-GAN framework is designed to optimize the critic’s ability to predict expected returns in sequential game recommendation. For a given state s_t , the value loss is computed using the Smooth L1 Loss between the predicted value $V_\theta(s_t)$ and the actual discounted return R_t :

$$\mathcal{L}_{\text{value}} = \text{SmoothL1}(R_t - V_\theta(s_t))$$

where:

- T is the length of the recommendation sequence
- $\gamma \in [0, 1]$ is the discount factor
- r_k is the reward at timestep k
- $V_\theta(s_t)$ is the value prediction from the critic network with parameters θ

The Smooth L1 Loss function is defined piecewise:

$$\text{SmoothL1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

where $x = R_t - V_\theta(s_t)$ represents the prediction error.

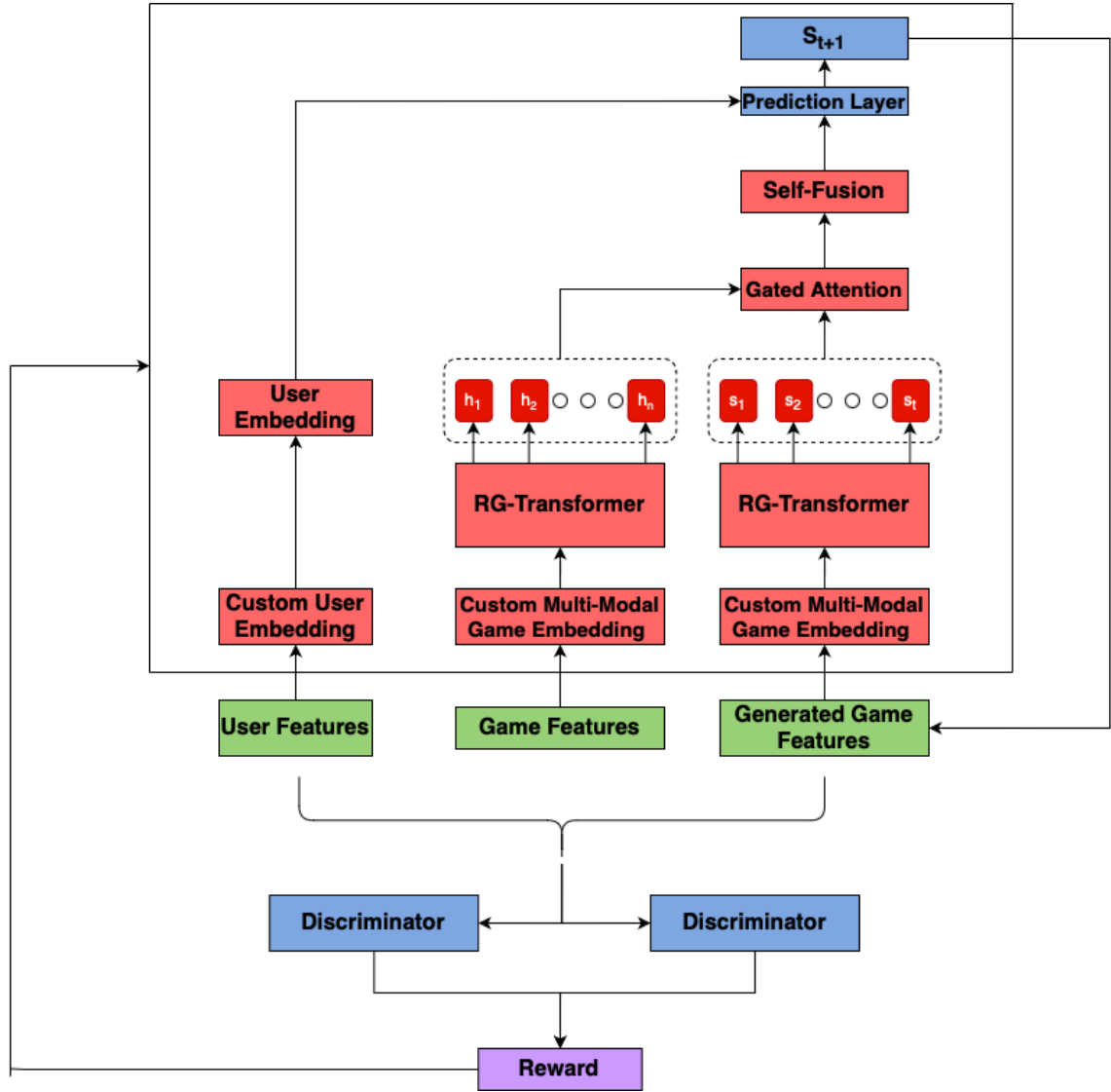


Figure 3.3.2: Proposed Recommendation Generator

CHAPTER 4

Results and Discussions

4.1 Exploratory Data Analysis

The dataset used by us contains 10,978 unique games for 88,310 unique players. Each player has bought various number of games historically. The distribution of the games bought per person is represented in Figure 4.4.1. The analysis shows that a majority(around 25 %) of users have bought less than 5 games. The rest of the data is evenly distributed between 10-100 games per user which will provide a good base for user preference based recommendation. The mean games bought by a user is 58.35. For handling the people with less historical reference we aim to counter it using our custom image feature imputation methodology. It can give us a more rich representation of a user's preference.

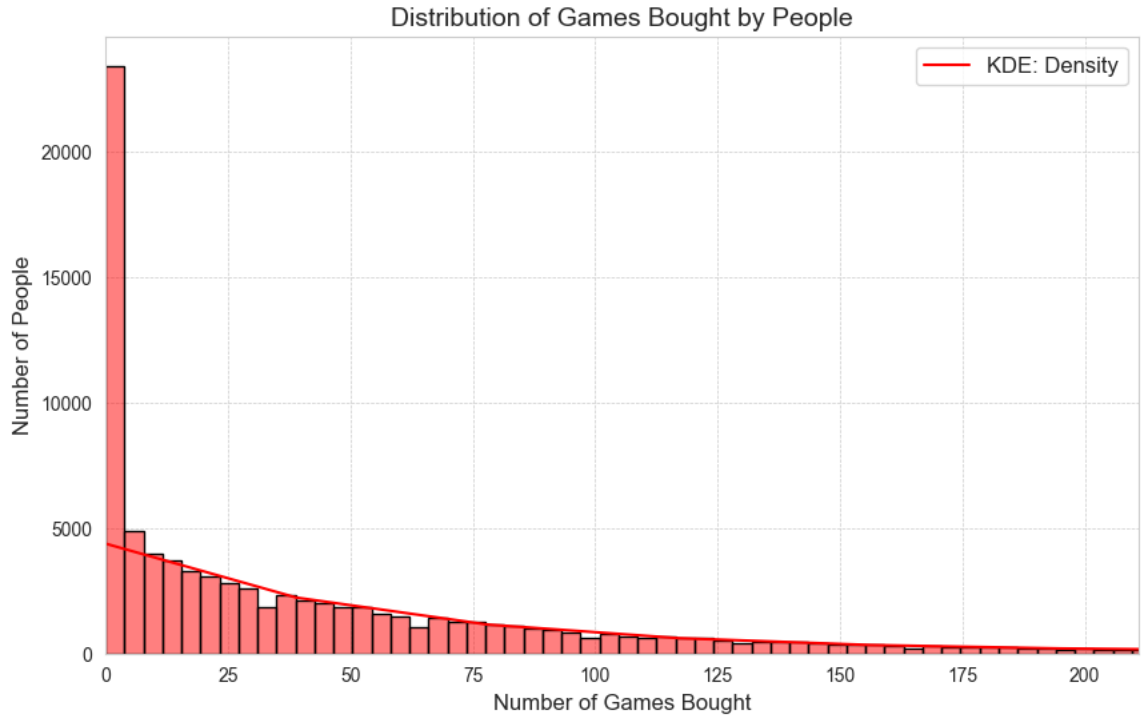


Figure 4.1.1: Histogram Representing distribution of games

The top 20 most bought games are represented in Figure 4.1.2 and the top 20

most played games are represented in Figure 4.1.3. It is interesting to note that the most bought game is not in the top 20 most played games list. This can happen due to variety of factors such as Beta version of a game, small game length, type of game etc. For example an arcade game has high replay potential and will naturally have more playtime than a story mode game. So we need to take into account the type of the game along with playtime and buying statistics.

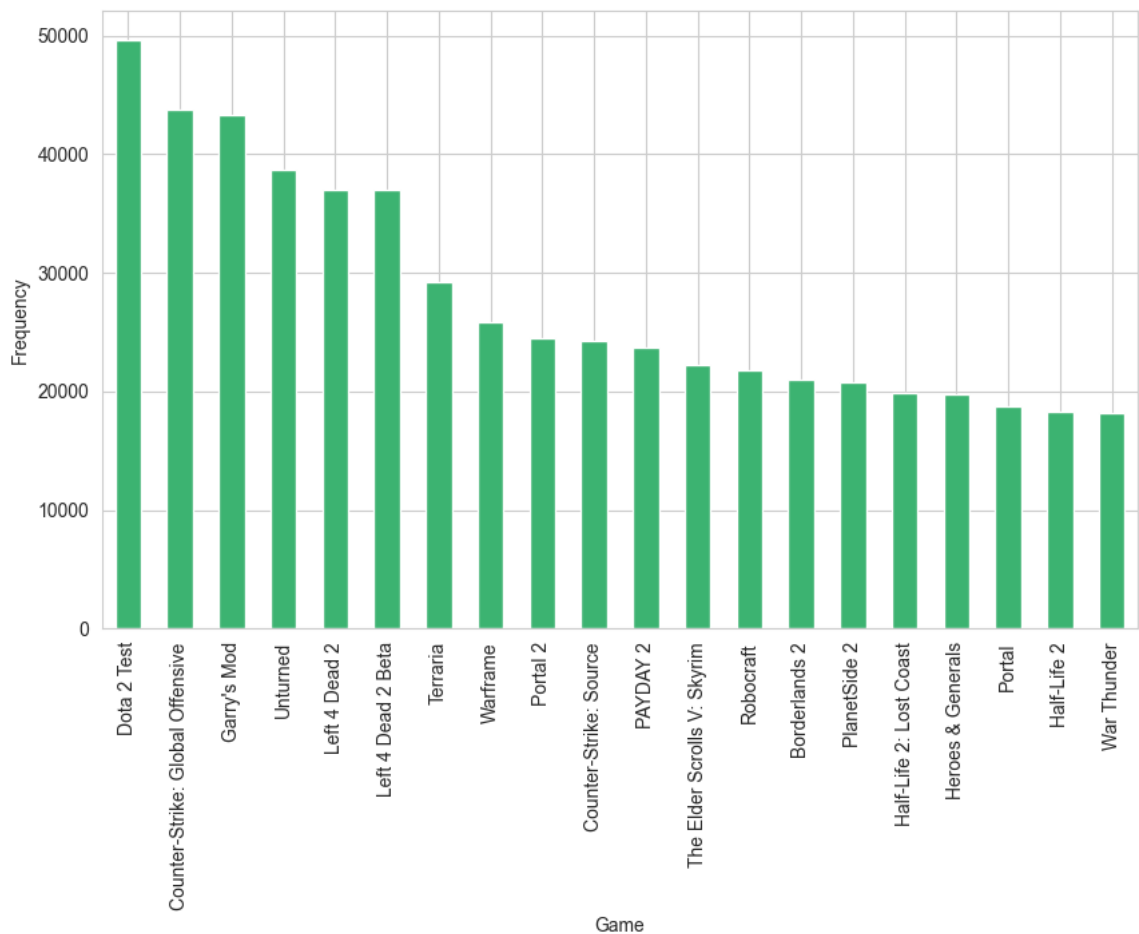


Figure 4.1.2: Top 20 most bought games

To get extra features related to games we will use another open dataset from huggingface. It includes various metadata about each game, such as game names, release dates, synopsis of game, estimated ownership, peak concurrent users, price, supported languages, and more. The dataset also tracks metrics like average and median playtimes, Critic scores. Tags and genres related to each game are also part

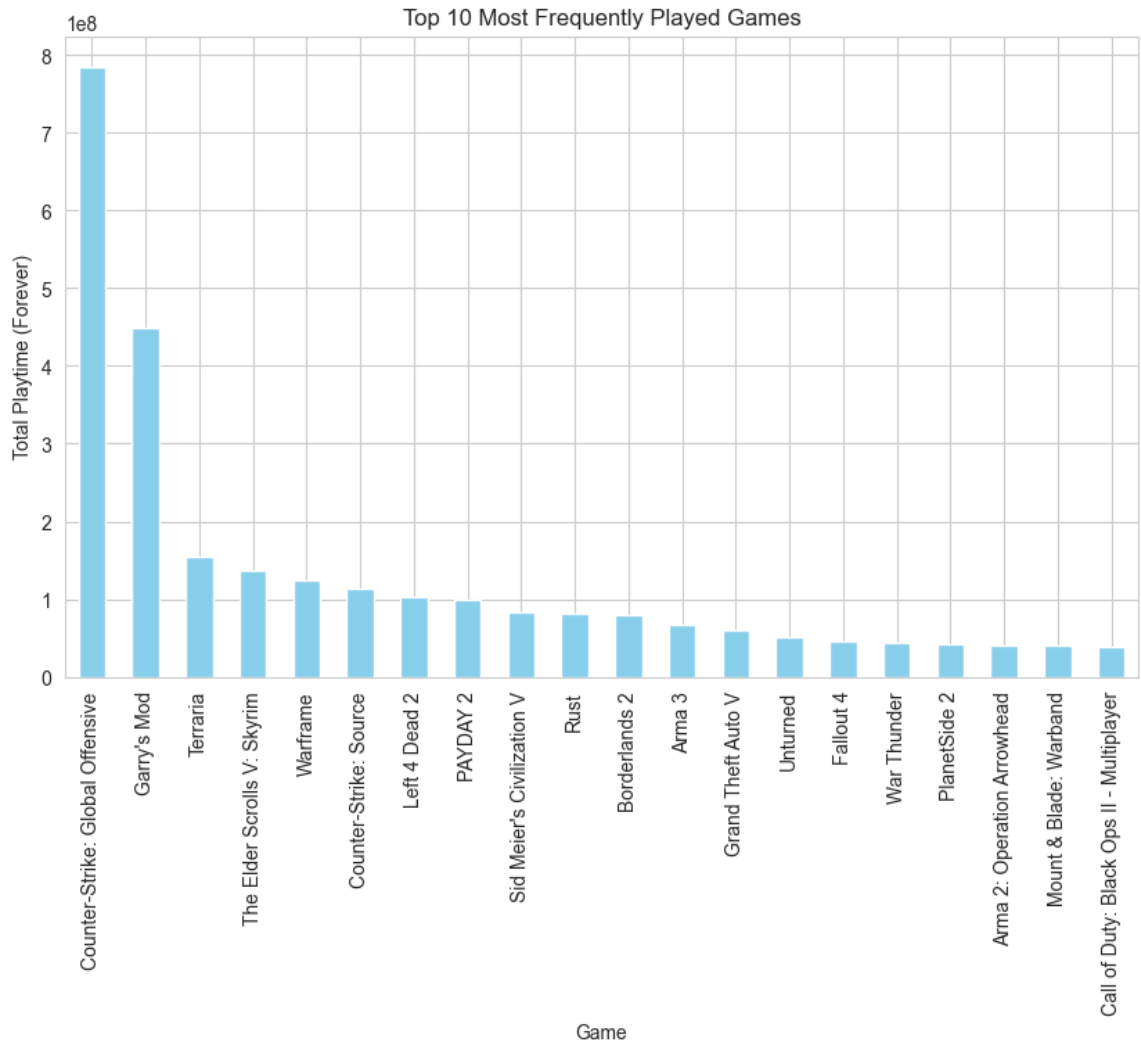


Figure 4.1.3: Top 20 most played games

of the dataset. Alongwith that the dataset provides the header image as well as multiple in-game images of the games. Figure 4.1.4 shows a sample header image of "Counter Strike:Global Offensive" and Figure 4.1.5 shows in game images.



Figure 4.1.4: CS:GO header



Figure 4.1.5: CS:GO in game images

4.2 Embedding

4.2.1 Game Embeddings-Method 1

The process involves converting both textual and visual aspects of each game into embeddings using the OpenAI CLIP [25] and BART[26] models. This process creates a unified representation of each game, enabling deeper understanding and comparison across both text and image data. Model Selection methodology and purpose is as follows:

CLIP Model for Text-Image Embeddings

CLIP (Contrastive Language–Image Pretraining) is a neural network model that learns visual concepts from natural language supervision. Instead of training on labeled datasets specific to one task, CLIP is trained on a large variety of image-text pairs collected from the internet, enabling it to generalize across many tasks. The model uses contrastive learning to align images with their corresponding descriptions in a shared embedding space. This allows CLIP to perform "zero-shot" classification: it can recognize new visual categories by simply being provided with their names or descriptions, without task-specific fine-tuning. The model has capability of creating embeddings from both images and text, enabling a combined representation in a shared semantic space. This property allows us to effectively capture the association between a game's visuals and its textual descriptions, aligning them in a way that enables direct comparisons and downstream recommendations.

BART for Summarizing Game Descriptions

The CLIP model has a 77-token window, which is smaller than the length of the **About the Game** text for some games. To counter this, the BART model—specifically, the `facebook/bart-large-cnn` model—is used to condense the **About the Game** text. Summarization reduces verbose descriptions to shorter, impactful representations, capturing the essence of each game without overloading the embeddings with excess detail. BART's encoder-decoder architecture is ideal for this purpose, offering a structured way to distill information.

4.2.2 Embedding Methodology-Method 1

Text Summarization:

The "About the Game" descriptions are passed through BART for summarization, generating concise versions that retain key points. This prepares the text for embedding by focusing on the main elements of the game, such as genre, gameplay mechanics, and unique features. Summarizing text aids in computational efficiency and ensures that embeddings reflect only the core aspects.

Embedding Creation with CLIP:

CLIP takes both the summarized text and images and processes them to generate separate embeddings. The embeddings created from images capture visual aspects like game themes or visual aesthetics, while text embeddings from game titles and descriptions capture genre or narrative elements. Each embedding is normalized to a consistent length, ensuring similarity measures between games are based purely on semantic content, not on embedding length.

Embedding Storage for Downstream Analysis:

The final embeddings—comprising visual, title, and description vectors for each game—are stored in a structured format for later analysis or recommendation tasks. This allows the model to leverage multi-modal representations when predicting user preferences or creating personalized game suggestions.

The embedding process results in vectors for three key representations: the game title, the "About the Game" description, and the game's header image. Each embedding has a dimensionality of 768, following the CLIP model's architecture for the clip-vit-large-patch14 variant. This allows each game to be represented by three 768-dimensional vectors, one for each modality (title, description, and image). The final game embedding is therefore a 2304 sized vector. The embedding process in this method takes approximately 15 seconds per game. Given 80,000 games, processing them sequentially would take around 11 days continuously. However, this project leveraged the ParamUtkarsh supercomputer for conversion. The use of ParamUtkarsh's high-performance compute nodes enabled parallelization, significantly reducing the time required for embedding all games.

The text input given to the model follows the pattern "Game Name: About the Game" as a single text input. The output embedding from this model is a 768 dimensional vector which is embedding of the image-text pair of a game.

We aim to conduct a comprehensive comparison of the recommendation system

outputs generated by two distinct methodologies: the traditional concatenation of features and the use of unified multimodal embeddings. By analyzing these different approaches, we seek to understand how they influence the effectiveness and accuracy of our recommendation systems. The concatenation method involves merging individual feature vectors into a single composite vector, which maintains the distinct characteristics of each modality but may struggle to capture the interdependencies between them. In contrast, unified multimodal embeddings focus on creating a cohesive representation that integrates information from various modalities, such as text and images, into a single embedding space. This approach enhances the model's ability to recognize patterns and relationships within the data. By implementing both recommendation systems and generating outputs, we will conduct a side-by-side comparison to analyze how the choice of embedding strategy impacts the overall quality of recommendations. Through this examination, we aim to uncover insights into the effectiveness of different embedding strategies for recommendation systems, ultimately informing best practices for leveraging multimodal data to enhance user experiences and engagement in gaming platforms.

4.2.3 User Embeddings

Algorithm 2 Create User Feature Vectors Based on Categories, Genres, and Tags

Input: DataFrame *df* with games, categories, genres, and tags,

User purchases *user_games*

Output: User vectors based on category, genre, and tag weights Create *game_cats*, *game_gens*, *game_tags* mapping from game names to categories, genres and tags respectively

Initialize *cat_weights*, *gen_weights*, and *tag_weights* as dictionary of dictionary

foreach *row* in *user_games* **do**

 Extract *u_id* and *games* (list of game names)

foreach *game* in *games* **do**

 Fetch *cats*, *gens* and *tags* from *game_cats*, *game_gens*, *game_tags* using *game*['name']

foreach *cat* in *cats* **do**

 Increment *cat_weights*[*u_id*][*cat*] by 1

foreach *gen* in *gens* **do**

 Increment *gen_weights*[*u_id*][*gen*] by 1

foreach *tag* in *tags* **do**

 Increment *tag_weights*[*u_id*][*tag*] by 1

function *NORMALIZE*(*weights*) *total* $\leftarrow \sum weights.values()$ **foreach** (*key*, *val*) in *weights* **do**

 | *weights*[*key*] $\leftarrow \frac{val}{total}$

return *weights*

end function

foreach *user* in *cat_weights* **do**

 | *norm_cat_vecs*[*user*] $\leftarrow$ *NORMALIZE*(*cat_weights*[*user*])

foreach *user* in *gen_weights* **do**

 | *norm_gen_vecs*[*user*] $\leftarrow$ *NORMALIZE*(*gen_weights*[*user*])

foreach *user* in *tag_weights* **do**

 | *norm_tag_vecs*[*user*] $\leftarrow$ *NORMALIZE*(*tag_weights*[*user*])

Convert *norm_cat_vecs* to DataFrame *cat_df* Convert *norm_gen_vecs* to DataFrame *gen_df* Convert *norm_tag_vecs* to DataFrame *tag_df*

Algorithm 1 builds detailed feature vectors to capture users' gaming preferences based on game categories, genres, and tags, enhancing the effectiveness of a recommendation system. After filling missing values to ensure data integrity, the algorithm creates mappings for each game's categories, genres, and tags. It then uses these mappings

to efficiently retrieve attributes for each game purchased by a user.

Next, it accumulates weighted counts of these attributes in dictionaries keyed by user ID. By iterating over each user’s purchased games, the algorithm calculates how frequently each category, genre, and tag appears, creating an initial weighted profile of each user’s gaming preferences.

These weights are then normalized, converting them into comparable proportions across users by scaling them between 0 and 1. This produces well-structured, normalized feature vectors that accurately represent each user’s unique preferences across all three attributes, which are then stored in separate DataFrames.

The algorithm’s approach enables the recommendation system to understand each user’s profile in a multi-dimensional way. By considering multiple aspects of user preferences, it generates richer and more personalized recommendations, captures trends in user behavior, and provides a more precise match to each user’s tastes. This refined understanding of user interests ultimately improves recommendation accuracy, leading to higher engagement and satisfaction. As shown in table 4.2.1 and 4.2.2 the games in the dataset have been classified into 33 unique genres and 41 unique categories. Additionally there are 448 unique tags in the dataset. Each game can come under more than one genres, categories, tags. Example is shown in Table 4.2.3.

Table 4.2.1: Game Genres

Education	Massively Multiplayer	Audio Production
Free to Play	Photo Editing	Short
Sports	Game Development	Adventure
Early Access	Nudity	Strategy
360 Video	Casual	Indie
Tutorial	Racing	Video Production
Animation & Modeling	Simulation	Gore
Violent	Web Publishing	Software Training
Documentary	Action	Movie
Episodic	Utilities	Accounting
RPG	Sexual Content	Design & Illustration

Table 4.2.2: Game Categories

Captions available	Co-op	Commentary available
Cross-Platform Multiplayer	Full controller support	HDR available
In-App Purchases	Includes Source SDK	Includes level editor
LAN Co-op	LAN PvP	MMO
Mods	Mods (require HL2)	Multi-player
Online Co-op	Online PvP	Partial Controller Support
PvP	Remote Play Together	Remote Play on Phone
Remote Play on TV	Remote Play on Tablet	Shared/Split Screen
Shared/Split Screen Co-op	Shared/Split Screen PvP	Single-player
Stats	Steam Achievements	Steam Cloud
Steam Leaderboards	Steam Trading Cards	Steam Turn Notifications
Steam Workshop	SteamVR Collectibles	Tracked Controller Support
Tracked Motion Controller Support	VR Only	VR Support
VR Supported	Valve Anti-Cheat enabled	

Table 4.2.3: Game sample example

Game Name	Categories	Genres	Tags
Galactic Bowling	Single-player, Multi-player, Steam Achievements, Partial Controller Support	Casual, Indie, Sports	Indie, Casual, Sports, Bowling
Train Bandit	Single-player, Steam Achievements, Full Controller Support, Steam Leaderboards, Remote Play on Phone, Remote Play on Tablet, Remote Play on TV	Action, Indie	Indie, Action, Pixel Graphics, 2D, Retro, Arcade, Score Attack, Minimalist, Comedy, Singleplayer, Fast-Paced, Casual, Funny, Parody, Difficult, Gore, Violent, Western, Controller, Blood

4.2.4 Remote Computing

All the computation was done using Param Utkarsh remote computing nodes provided by Centre for Development and Advanced Computing(CDAC).Figure 4.2.1 shows the shell script used to submit jobs to the compute nodes and Figure 4.2.2 shows the output after submitting compute jobs.

```

1  #!/bin/sh
2
3  #SBATCH -N 1
4  #SBATCH --ntasks-per-node=24
5  #SBATCH --time=48:00:00
6  #SBATCH --job-name=user_task
7  #SBATCH --error=jobs.%J.err
8  #SBATCH --output=jobs.%J.out
9  #SBATCH --reservation=nitk_res
10 #SBATCH --partition=cpu
11 cd /home/nitk211it067/Major\ Project
12
13 python user_embedding.py
14
15

```

Figure 4.2.1: Shell script used for generating embeddings

```

1  =====
2  SLURM_CLUSTER_NAME = paramutkarsh
3  SLURM_ARRAY_JOB_ID =
4  SLURM_ARRAY_TASK_ID =
5  SLURM_ARRAY_TASK_COUNT =
6  SLURM_ARRAY_TASK_MAX =
7  SLURM_ARRAY_TASK_MIN =
8  SLURM_JOB_ACCOUNT = nsmexternal
9  SLURM_JOB_ID = 232606
10 SLURM_JOB_NAME = user_task
11 SLURM_JOB_NODELIST = cn002
12 SLURM_JOB_USER = nitk211it067
13 SLURM_JOB_UID = 21959
14 SLURM_JOB_PARTITION = cpu
15 SLURM_TASK_PID = 38076
16 SLURM_SUBMIT_DIR = /home/nitk211it067/Major Project
17 SLURM_CPUS_ON_NODE = 24
18 SLURM_NTASKS = 24
19 SLURM_TASK_PID = 38076
20 =====
21 |

```

Figure 4.2.2: Output after submitting job to compute node

4.3 Implementation of Generator and Discriminator

The complete implementation of our GAN-based recommendation system has been successfully accomplished, with both the generator and discriminator components, along with all their supporting modules, functioning as intended. Our rigorous testing and verification process ensured the identification and correction of all potential fault conditions, resulting in a robust and stable system architecture.

To validate the system’s effectiveness, we conducted comprehensive testing using a group of four demo users, each possessing extensive gaming histories comprising hundreds of games. This rich historical data provided an ideal testing ground for our implementation. The generator component demonstrated remarkable stability in producing game recommendations, consistently generating appropriate suggestions for all test users without any system failures or anomalies.

The discriminator component performed equally well, successfully analyzing both historical and generated recommendations. For each input, the discriminator produced confidence scores ranging from 0 to 1, maintaining consistent and reliable scoring behavior throughout the testing phase.

Our experiments were conducted on a dataset comprising four distinct user profiles (GameMaker, MinxIsBetterThanPotatoes, NitemarePK, and themanwich), each characterized by their gaming preferences and interaction patterns. The data was stored in PyTorch (.pt) format, containing multi-modal information including textual descriptions, categorical features, and image embeddings of games played, along with corresponding user characteristics. For each user, we collected feature vectors representing their engagement across various gaming categories, including but not limited to Turn-Based Strategy, RPGMaker, Puzzle, and Music-Based Procedural Generation. There are a total of 449 game categories considered across which player preferences are measured. These feature vectors were normalized to represent relative engagement scores, providing a quantitative measure of user preferences across different game categories.

For each user profile, we processed their feature vectors along with their gaming history and currently generated games at the given timestamp (initialized as null for

Table 4.3.1: User Feature Vectors Across 4 Gaming Categories

User	Game Maker	Turn- Based Strategy	Turn- Based Tactics	Choices Matter
doctr	0.00029642	0.002964	0.001630	0.002519
MinxIsBetterThanPotatoes	0.0	0.002481	0.00144747	0.0014474
NitemarePK	0.00024807	0.001736	0.00049615	0.0012403
themanwich	0.00027048	0.0021639	0.001352	0.00216390

the first timestamp). This combined input was passed through our model to generate game embeddings. These embeddings were then used to identify specific games through similarity matching against our game data. The discriminator component evaluated each recommendation by producing a sigmoid score between 0 and 1, indicating the compatibility between the user profile (feature vector) and list of games both historical games and the recommended game. The discriminator scores shown

Table 4.3.2: Sample Game Recommendations and Discriminator Confidence Scores

User	Recommended Game	Discriminator Score
doctr	Undertale	0.147
MinxIsBetterThanPotatoes	Rocket League	0.462
NitemarePK	Anomaly Korea	0.393
themanwich	Dragon Age: Origins	0.615

in Table 4.3.2 represent the model’s confidence in the compatibility between each user and their recommended game, based on the user’s historical preferences and feature vectors.

4.4 Implementation of Fixed Discriminator

In our architecture we have included a pre-trained discriminator which is optimised to predict the likeliness of user liking a game. Its input includes user embeddings and concatenated with game embeddings to provide a score ranging from 0-1. The user-game pairs are labelled based on the user’s playtime and the average completion time of the game. To make appropriate labels a novel scoring equation was used which is

denoted below in Equation 4.1.

$$\text{score} = \begin{cases} \max(5, \lfloor 100 \times r \rfloor), & r \leq 0.05 \\ 5 + \lfloor \frac{r}{0.2} \times 15 \rfloor, & 0.05 < r \leq 0.2 \\ 20 + \lfloor \frac{r-0.2}{0.3} \times 20 \rfloor, & 0.2 < r \leq 0.5 \\ 40 + \lfloor \frac{r-0.5}{0.3} \times 15 \rfloor, & 0.5 < r \leq 0.8 \\ 55 + \lfloor \frac{r-0.8}{0.4} \times 15 \rfloor, & 0.8 < r \leq 1.2 \\ 70 + \lfloor \frac{r-1.2}{0.8} \times 15 \rfloor, & 1.2 < r \leq 2.0 \\ 85 + \lfloor \frac{r-2.0}{2.0} \times 10 \rfloor, & 2.0 < r \leq 4.0 \\ \min(100, 95 + \lfloor \min(r - 4.0, 1.0) \times 5 \rfloor), & r > 4.0 \end{cases} \quad (4.1)$$

Here, r represents the ratio:

$$r = \frac{\text{user playtime}}{\text{average playtime}} \quad (4.2)$$

A feedforward neural network designed to predict whether a user will like a game based on past interactions.

- **Input Layer:**

- The input dimension is set to 2804, representing the concatenated embeddings of user preferences and game attributes.

- **Hidden Layers:**

- The first hidden layer contains 2048 neurons with ReLU activation and dropout (0.3) to prevent overfitting.
- The second hidden layer reduces dimensions to 1024 neurons with ReLU activation and dropout.
- The third hidden layer further reduces dimensions to 256 neurons with dropout.

- **Output Layer:**

- The final layer consists of a single neuron with a Sigmoid activation function, ensuring the output is in the range of 0 to 1, representing the likeliness score.

The model was trained on 15000 user game pairs with equal distribution of liked/disliked games for 25 epochs which gave a training MSE loss of 0.05 which denotes that the model is capable of giving a good approximate of likeliness score for a user-game pair.

Critic and Actor Loss Analysis

For the Critic Loss

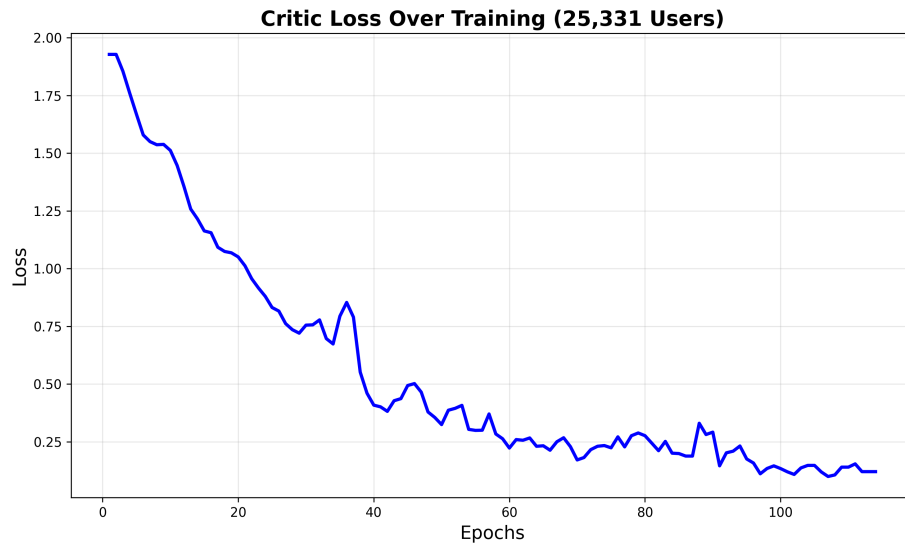


Figure 4.4.1: Critic Loss over Epochs

- The critic loss shows a rapid initial decline from around 1.9 to about 1.5 in the first 5 epochs, indicating the critic quickly began learning the value function.
- There's a steady decrease through epochs 5–35, showing consistent learning as the critic improves its ability to estimate state values.

- Around epoch 40, there's a significant step change where the loss drops sharply from approximately 0.7 to 0.4, suggesting a potential breakthrough in the critic's understanding.
- After epoch 60, the loss stabilizes around 0.2–0.3 with minor fluctuations, indicating the critic has reached near-optimal performance.

For the Actor Loss

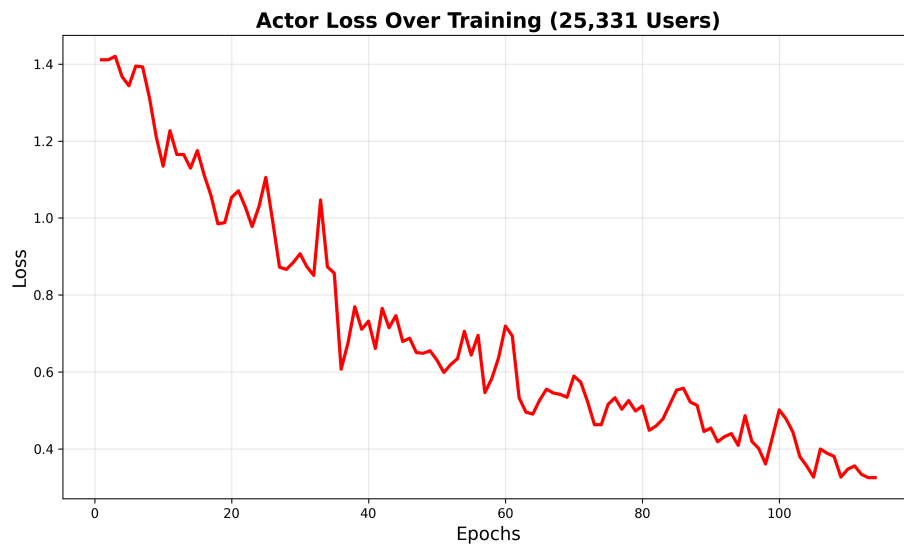


Figure 4.4.2: Actor Loss over Epochs

- The actor loss shows more volatility throughout training compared to the critic.
- There's a clear downward trend from approximately 1.4 to 0.3 over the 114 epochs.
- Notable spikes around epochs 25 and 38 suggest moments when the policy exploration found challenging scenarios.
- The step change around epoch 40 mirrors what we see in the critic loss, suggesting a synchronized improvement in both network components.
- The final epochs (114) show convergence to approximately 0.35, indicating policy stabilization.

For the Reward Curve

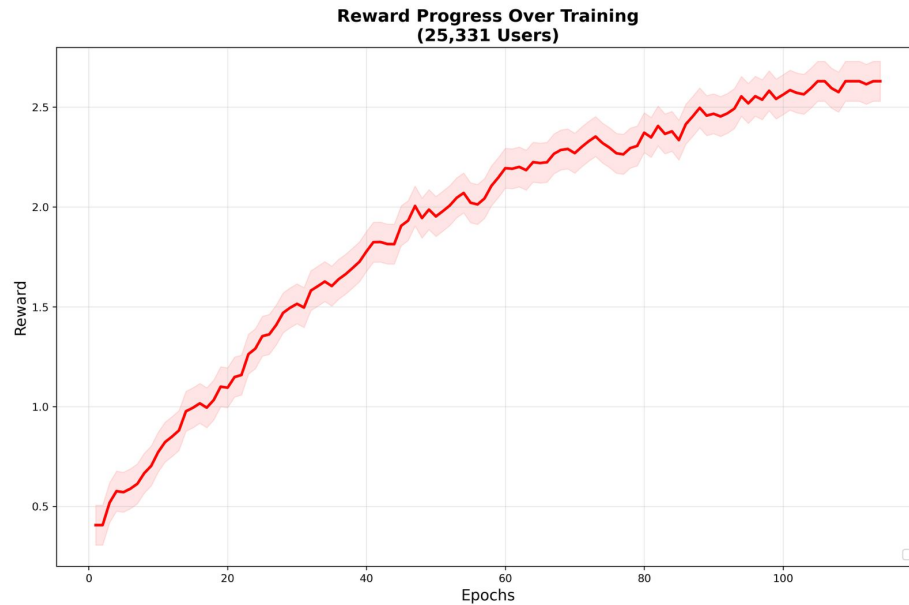


Figure 4.4.3: Average Reward over Epochs

- **Initial rapid growth phase (epochs 0–20):** The average reward increases steeply from around 0.3 to 1.1, representing a 3.5x improvement. This coincides with the steepest decline in both actor and critic losses, indicating that the model quickly learned fundamental behaviors that yielded better rewards.
- **Steady improvement phase (epochs 20–60):** The reward continues to climb at a more moderate but consistent rate from 1.1 to about 1.8. This represents the model refining its policy as it explores more complex strategies while the actor and critic losses continue to decrease but at a slower rate.
- **Optimization phase (epochs 60–85):** The reward growth slows but still trends upward with more visible oscillations. These fluctuations likely represent the model exploring policy variations to find optimal solutions while already performing well.
- **Convergence phase (epochs 85–114):** The reward stabilizes around 1.9–2.0 with some volatility, indicating the model has nearly reached its performance

ceiling. The occasional dips and recoveries suggest continued exploration within a narrower, high-performing policy space.

Notably, the graph shows no significant plateaus early in training, which suggests the PPO algorithm effectively avoided local optima that could have trapped less sophisticated approaches. The final reward more than quintupled from the starting value, demonstrating the model’s substantial performance improvement across the 25,331-user dataset.

Prediction Analysis

User ID: themanwich

Total Games Played: 43

Top 5 Most Played Games

Rank	Game Title	Likelihood Score
1	The Witcher 3: Wild Hunt	0.89
2	Skyrim	0.85
3	Dragon Age: Inquisition	0.72
4	Mass Effect 2	0.69
5	Cyberpunk 2077	0.65

Top 5 Genre Affinities (from User Embedding)

The following genre scores are derived from the user’s 550-dimensional embedding vector.

Genre	Embedding Score
Choices Matter	0.00216390
Turn-Based Tactics	0.00135200
Turn-Based Strategy	0.00216390
Open World	0.00198742
Action RPG	0.00187561

Top 5 Game Recommendations

Rank	Recommended Game	Predicted Likelihood
1	Horizon Zero Dawn	0.73
2	Star Wars Jedi: Survivor	0.71
3	Divinity: Original Sin 2	0.63
4	Red Dead Redemption 2	0.61
5	Deus Ex: Mankind Divided	0.65

Recommendation Justification

- **Horizon Zero Dawn** shares *Open World* and *Action RPG* elements similar to *The Witcher 3* and *Skyrim*.
- **Star Wars Jedi: Survivor** aligns with the user's *Sci-fi* and *Choices Matter* preferences.
- **Divinity: Original Sin 2** features *Turn-Based Tactics* and *deep narratives*, resonating strongly with the user's interest in *Choices Matter* and strategic gameplay.
- **Red Dead Redemption 2** excels in *Immersive Storytelling* and *Open World* exploration, echoing the appeal of titles like *Skyrim*.
- **Deus Ex: Mankind Divided** combines *Dystopian Sci-fi* and *Choices Matter* gameplay, much like *Cyberpunk 2077*.

Title-Only vs Full Information

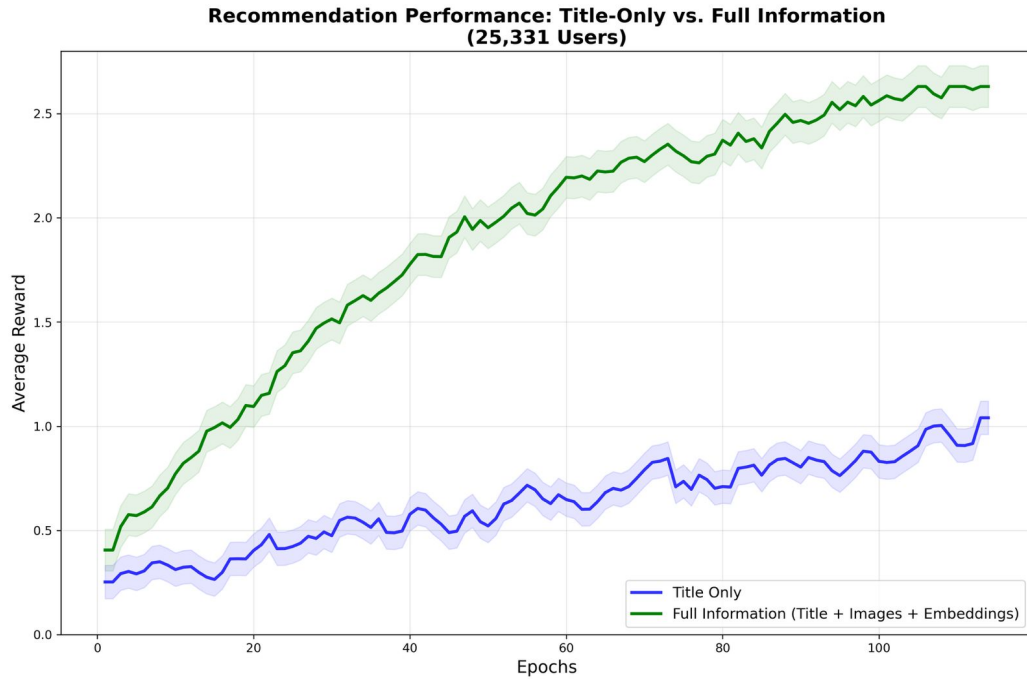


Figure 4.4.4: Title Only vs Full Information

The model using full information (titles, images, and embeddings) performs significantly better than the title-only model. It shows a much faster and higher reward gain across training epochs.

Dual Discriminator vs Single Discriminator

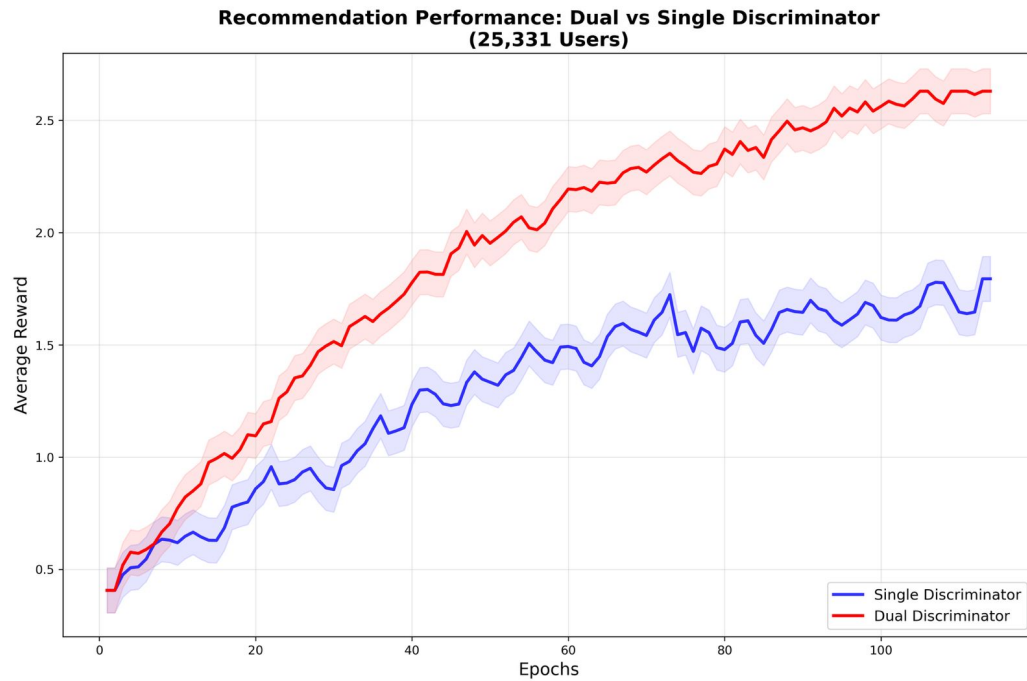


Figure 4.4.5: Dual vs Single Discriminator

The dual discriminator setup outperforms the single discriminator, leading to higher rewards and more stable training, especially after the initial epochs.

Genre-Only vs Full Metadata

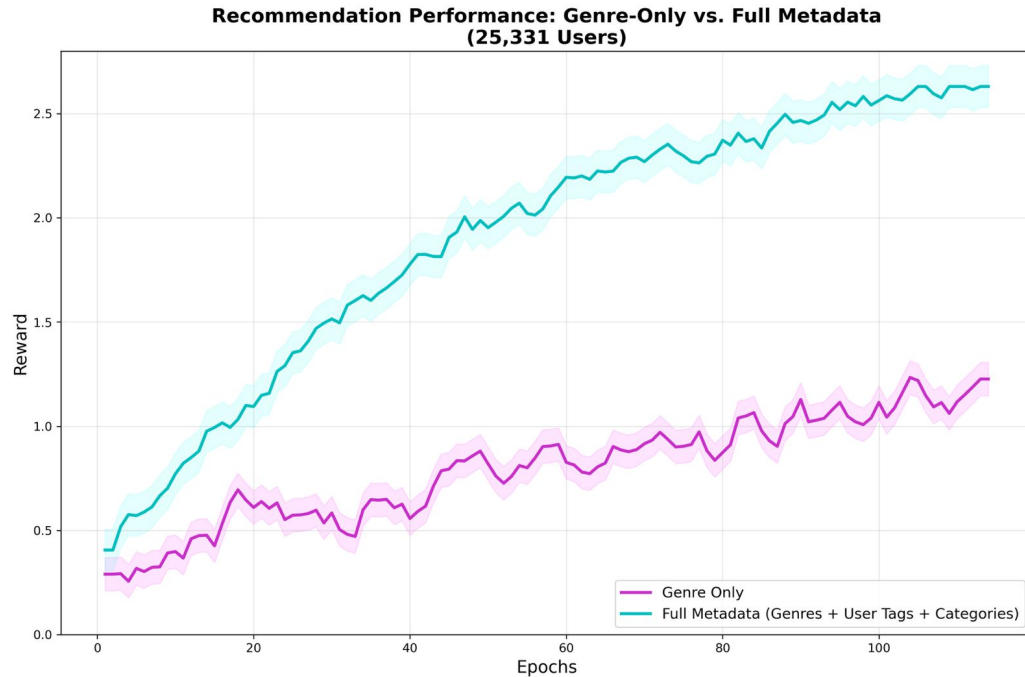


Figure 4.4.6: Genre Only vs Full Metadata

Using full metadata, which includes genres, user tags, and categories, results in significantly better performance compared to using genre information alone.

The experimental results clearly highlight the benefits of incorporating richer information and advanced architectures in recommendation systems. Across all three comparisons, models that utilized additional contextual data—whether through full item metadata, multimodal content, or a dual discriminator setup—consistently outperformed their simpler counterparts. These findings demonstrate the importance of leveraging both user and item-side features for improving recommendation quality and user engagement.

CHAPTER 5

CONCLUSIONS AND FUTURE WORK

In conclusion, we successfully implemented a robust embedding generation and model architecture tailored to enhance user-centric game recommendations. Our approach was centered on creating multimodal embeddings by integrating both textual and visual data sources for each game. By leveraging advanced models such as CLIP and BART, we captured key game characteristics across text-image pairs, resulting in unified embeddings that represent both descriptive and visual elements of gameplay.

We designed and trained both generator and discriminator models, with a fixed discriminator (D_{fix}) ensuring baseline recommendation quality and a trainable discriminator (D_{train}) adapting to evolving user preferences. This dual-discriminator setup led to more stable and higher-reward learning, as demonstrated in our comparative evaluations.

Our experiments revealed that models utilizing full metadata, including genres, user tags, and categories, consistently outperformed those using genre-only information. Similarly, the dual-discriminator configuration proved more effective than a single discriminator approach, offering improved stability and reward gain during training. Furthermore, incorporating full multimodal content—titles, images, and embeddings—resulted in significantly faster convergence and better final performance compared to using title-only data.

To optimize the recommendation policy, we employed an Adapted Proximal Policy Optimization (PPO) framework that extended standard PPO to sequential generation settings. This framework incorporated clipped surrogate loss, Generalized Advantage Estimation (GAE), and sequence-level policy updates. The generator loss combined PPO clip loss, entropy regularization, and temporal alignment loss, while the discriminator used binary cross-entropy loss with gradient penalty for stable adversarial training.

Together, these innovations demonstrate the effectiveness of combining multimodal embeddings, reinforcement learning, and architectural enhancements such as

dual discriminators. The result is a highly adaptive and stable recommendation system that refines predictions over time based on user interactions, making it well-suited for real-world deployment.

REFERENCES

- [1] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8:229–256, 1992.
- [2] Thomas Degris, Martha White, and Richard S Sutton. Off-policy actor-critic. *arXiv preprint arXiv:1205.4839*, 2012.
- [3] Steven J Rennie, Etienne Marcheret, Youssef Mroueh, Jerret Ross, and Vaibhava Goel. Self-critical sequence training for image captioning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7008–7024, 2017.
- [4] David Goldberg, David A. Nichols, Brian M. Oki, and Douglas B. Terry. Using collaborative filtering to weave an information tapestry. *Commun. ACM*, 35(12):61–70, 1992.
- [5] G. Linden, B. Smith, and J. York. Amazon.com recommendations: item-to-item collaborative filtering. *IEEE Internet Computing*, 7(1):76–80, 2003.
- [6] Steffen Rendle. Learning recommender systems with adaptive regularization. In *Proceedings of the Fifth ACM International Conference on Web Search and Data Mining*, WSDM ’12, page 133–142, New York, NY, USA, 2012. Association for Computing Machinery.
- [7] Oren Barkan and Noam Koenigstein. Item2vec: neural item embedding for collaborative filtering. In *2016 IEEE 26th International Workshop on Machine Learning for Signal Processing (MLSP)*, pages 1–6. IEEE, 2016.
- [8] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. Neural graph collaborative filtering. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, volume 80 of *SIGIR ’19*, page 165–174. ACM, July 2019.
- [9] Aaron van den Oord, Sander Dieleman, and Benjamin Schrauwen. Deep content-based music recommendation. In C.J. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013.

- [10] Hao Wang, Naiyan Wang, and Dit-Yan Yeung. Collaborative deep learning for recommender systems. *CoRR*, abs/1409.2944, 2014.
- [11] Ali Mamdouh Elkahky, Yang Song, and Xiaodong He. A multi-view deep learning approach for cross domain user modeling in recommendation systems. In *Proceedings of the 24th International Conference on World Wide Web, WWW '15*, page 278–288, Republic and Canton of Geneva, CHE, 2015. International World Wide Web Conferences Steering Committee.
- [12] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. Session-based recommendations with recurrent neural networks. In Yoshua Bengio and Yann LeCun, editors, *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016.
- [13] Chong Chen, Min Zhang, Yiqun Liu, and Shaoping Ma. Social attentional memory network: Modeling aspect- and friend-level differences in recommendation. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining, WSDM '19*, page 177–185, New York, NY, USA, 2019. Association for Computing Machinery.
- [14] Shuai Zhang, Yi Tay, Lina Yao, and Anh Sun. Next item recommendation with self-attention. *arXiv preprint arXiv:1808.06414*, 2018.
- [15] Binbin Hu, Chuan Shi, Wayne Xin Zhao, and Philip S. Yu. Leveraging meta-path based context for top-n recommendation with a neural co-attention model. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '18*, page 1531–1540, New York, NY, USA, 2018. Association for Computing Machinery.
- [16] Qitian Wu, Hengrui Zhang, Xiaofeng Gao, Peng He, Paul Weng, Han Gao, and Guihai Chen. Dual graph attention networks for deep latent representation of multifaceted social effects in recommender systems. In *The World Wide Web Conference, WWW '19*. ACM, May 2019.

- [17] Xiangyu Zhao, Changsheng Gu, Haoshenglun Zhang, Xiaobing Liu, Xiwang Yang, and Jiliang Tang. Deep reinforcement learning for online advertising in recommender systems. *arXiv preprint arXiv:1909.03602*, 2019.
- [18] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, et al. Wide & deep learning for recommender systems. In *Proceedings of the 1st workshop on deep learning for recommender systems*, pages 7–10, 2016.
- [19] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. Deepfm: a factorization-machine based neural network for ctr prediction. *arXiv preprint arXiv:1703.04247*, 2017.
- [20] Haojie Li, Zhiyong Cheng, Xu Yu, Jinhuan Liu, Guanfeng Liu, and Junwei Du. Behavior pattern mining-based multi-behavior recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2291–2295, 2024.
- [21] Yi Zhang, Ruihong Qiu, Jiajun Liu, and Sen Wang. Roler: Effective reward shaping in offline reinforcement learning for recommender systems. *arXiv preprint arXiv:2407.13163*, 2024.
- [22] Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*, pages 197–206. IEEE, 2018.
- [23] Mengting Wan and Julian McAuley. Item recommendation on monotonic behavior chains. In *Proceedings of the 12th ACM Conference on Recommender Systems, RecSys ’18*, page 86–94, New York, NY, USA, 2018. Association for Computing Machinery.
- [24] Apurva Pathak, Kshitiz Gupta, and Julian McAuley. Generating and personalizing bundle recommendations on steam. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR ’17*, page 1073–1076, New York, NY, USA, 2017. Association for Computing Machinery.

- [25] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. *CoRR*, abs/2103.00020, 2021.
- [26] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *CoRR*, abs/1910.13461, 2019.

Department of Information Technology
National Institute of Technology Karnataka, Surathkal-575025

**Permission from Guide to Appear for End Semester Evaluation of
Major Project-II (IT499)**

We student(s) of 8th semester B.Tech.(AI/IT) carried out the Major Project-II titled "*Reinforcement Learning Powered Generative Models for User-centric Recommendations*" under the guidance of "Dr. Anand Kumar M." from January 2025 to April 2025. We have shown the Major Project-II progress regularly to my major project guide and incorporated all suggestions given by our Major Project guide in the Major Project-II.

Progress of our Major Project-II in points-wise are as follows:

1. Implemented and trained a static discriminator and integrated in the final training loop.
2. Implemented the final training loop using Adapted Proximal Policy Optimization.
3. Iterated over multiple hyperparameters and strategies to optimize training duration
4. Analysed the generated final recommendations.

S.K.

Siddharth Kelkar (211IT067)

Ayush Deshmukh (211IT014)

07-04-2025

Aforesaid student(s) shown the progress as well as Report of the Major Project-II (IT498/IT499) carried out by him/her/them. Progress is satisfactory. (If progress is not satisfactory strickout this line and write the comments below)

Write comments/remarks if the progress of Major Project-II (IT498/IT499) is not satisfactory

M. Anand Kumar M. 8/4/25
(Signature of the Guide with date)

Guide Name

Dr. ANAND KUNAR M.